

# Tiny Intrusion Detection System (tIDS): A Behavioral Machine Learning Approach

Aya Mouffok  
*AI Engineering*  
*ENSIA*  
Algiers, Algeria

Mohamed Al Amin Saad  
*AI Engineering*  
*ENSIA*  
Algiers, Algeria

Ines Sekfali  
*3rd year student*  
*ENSIA*  
Algiers, Algeria

Malak Meliani  
*AI Engineering*  
*ENSIA*  
Algiers, Algeria

Hanaa Chahbar  
*AI Engineering*  
*ENSIA*  
Algiers, Algeria

**Abstract**—Traditional signature-based Intrusion Detection Systems (IDS) frequently fail to identify zero-day vulnerabilities and custom-crafted network attacks. This paper presents the development of a tiny Intrusion Detection System (tIDS) powered by behavioral machine learning. Utilizing a custom air-gapped network environment, live-fire attacks including Smurf floods, TCP SYN floods, and ARP/DNS spoofing were generated. The resulting PCAP traffic was processed and engineered into a 35-feature dataset. Evaluation of Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and Random Forest classifiers demonstrates that the Random Forest architecture achieves a 99% overall accuracy in detecting anomalous network behavior.

**Index Terms**—Intrusion Detection, Machine Learning, Random Forest, Network Security, ENSIA

## I. INTRODUCTION (PROBLEMATIC)

The rapid expansion of interconnected digital infrastructures has fundamentally elevated the importance of robust network security. Central to defending these environments are Intrusion Detection Systems (IDS), which traditionally rely on signature-based detection mechanisms. While effective against well-documented threats, traditional signature-based IDSs inherently fail when confronted with zero-day vulnerabilities, obfuscated malware, and custom-crafted network attacks. Because these legacy systems require a pre-existing database of known attack signatures, they operate reactively rather than proactively. Consequently, they are persistently blind to novel attack vectors and struggle to adapt to dynamic, evolving network topologies.

To address these critical limitations, modern network security paradigms are shifting toward behavioral machine learning approaches. Rather than matching rigid payload signatures, behavioral anomaly detection establishes a statistical baseline of normal network traffic and continuously analyzes incoming packet flows for deviations. This methodology forms the core of the tiny Intrusion Detection System (tIDS) proposed in this paper.

This project focuses on evaluating the tIDS against a diverse spectrum of sophisticated link-layer, network-layer, and transport-layer attacks. Specifically, the system is engineered and tested to detect seven targeted attack vectors: (1) Spanning Tree Protocol (STP) spoofing attacks, (2) Address Resolution Protocol (ARP) poisoning, (3) Domain Name System (DNS) spoofing, (4) TCP SYN floods, (5) Smurf attacks, (6) Internet

Group Management Protocol (IGMP) spoofing, and (7) UDP flooding.

By simulating these specific threats within a controlled, air-gapped environment, we demonstrate the efficacy of machine learning classifiers—most notably the Random Forest architecture—in identifying complex intrusions based solely on behavioral traffic features.

The remainder of this paper is structured as follows: Section II details the threat model, the air-gapped attack simulation, and the feature extraction process. Section III outlines the proposed behavioral machine learning architecture. Section IV describes the system implementation, including real-time traffic sniffing and the user interface. Finally, Section V presents the evaluation metrics and performance results, followed by concluding remarks in Section VI.

## II. THREAT MODEL & DATA ENGINEERING

### A. Air-Gapped Environment & Attack Simulation

To ensure safe, controlled, and reproducible experimentation without risk of unintended network interference or security incidents, all attack simulations were conducted within a strictly air-gapped virtual environment. This isolated network topology prevented external connectivity, allowing us to generate high-fidelity attack traffic while maintaining complete separation from production systems.

1) *Virtual Network Infrastructure*: The experimental environment was constructed using VirtualBox virtualization, establishing a closed network with multiple interconnected virtual machines. The infrastructure comprised the following components:

- **Attacker Machine**: Kali Linux with pre-installed security tools (Ettercap, Yersinia, custom packet generators)
- **Target Machines**: Debian/Ubuntu Linux systems with minimal hardening to represent vulnerable endpoints
- **Network Topology**: A simulated LAN with realistic routing and switching behavior, enabling accurate layer-2 and layer-3 attack simulation
- **Traffic Capture Node**: A dedicated machine running Wireshark in passive monitor mode

This configuration enabled the generation of realistic network traffic patterns while maintaining deterministic control over attack parameters and timing.

2) *Network-Layer Attack Scenarios*: The following link-layer and network-layer attacks were systematically executed and captured:

a) *ARP Poisoning (ARP Spoofing)*: Address Resolution Protocol (ARP) poisoning attacks were conducted using Ettercap to establish a Man-in-the-Middle (MITM) position between target hosts and the default gateway. The attacker crafted forged ARP replies associating its MAC address with legitimate IP addresses (particularly the gateway IP). This technique enabled traffic interception and selective packet forwarding. Attack success was validated through analysis of captured ARP traffic, which exhibited multiple conflicting IP-to-MAC bindings and abnormal ARP reply patterns inconsistent with normal network behavior.

b) *DNS Spoofing (DNS Hijacking)*: Domain Name System (DNS) spoofing was executed using Ettercap's DNS spoofing plugin in conjunction with ARP poisoning. The attacker intercepted legitimate DNS queries and responded with forged answers, redirecting victims toward attacker-controlled hosts or arbitrary malicious IP addresses. This attack exploits DNS's inherent lack of authentication mechanisms and relies on IP address spoofing at the network layer. Captured PCAP files exhibited characteristic patterns: DNS query packets followed by responses with incongruent domain-to-IP mappings and timing anomalies indicative of spoofed replies.

c) *STP Spoofing (Spanning Tree Protocol Attacks)*: Spanning Tree Protocol (STP) spoofing attacks were simulated within a GNS3-based network topology to evaluate the system's sensitivity to layer-2 topology manipulation. The attacker injected malicious Bridge Protocol Data Unit (BPDU) packets claiming superior bridge priority to manipulate the spanning tree election process. If successful, the attacker could become the root bridge, positioning itself to intercept all inter-switch traffic. Attack execution was validated through observation of abnormal BPDU message patterns, rapid topology changes, and convergence events not typical of stable networks.

3) *Transport-Layer and Volumetric Attack Scenarios*: To simulate denial-of-service and resource exhaustion threats, the following attacks were generated:

a) *TCP SYN Flood (SYN Flood DoS)*: The TCP SYN flood attack exploits the TCP three-way handshake protocol. The attacker transmits a high volume of TCP SYN packets to target services, typically on port 80 (HTTP) or other listening ports, without completing the connection establishment process. The target allocates kernel resources (memory, socket structures) for each half-open connection. When the queue of pending connections is exhausted, the server becomes unable to accept legitimate connection requests. In the captured traffic, this attack manifests as a dramatic spike in SYN packets originating from multiple source IPs (spoofed), coupled with an absence of corresponding ACK packets or significantly reduced ACK response rates compared to benign traffic.

b) *Smurf Attack (ICMP Amplification)*: The Smurf attack is an ICMP-based amplification distributed denial-of-service (DDoS) attack. The attacker sends ICMP Echo Request (ping)

packets to a broadcast address (e.g., 192.168.1.255) while spoofing the victim's IP address as the source. All hosts on the broadcast network respond with ICMP Echo Reply packets, overwhelming the victim with traffic. In this attack, the attacker amplifies its traffic by a factor of  $N$ , where  $N$  is the number of responsive hosts. Packet captures reveal a characteristic pattern: a single (or few) ICMP Echo Request packets followed by a massive flood of ICMP Echo Reply packets, all directed toward the spoofed victim IP address.

c) *UDP Flood (User Datagram Protocol Flood)*: The UDP flood attack is a volumetric denial-of-service attack that overwhelms a target with a high volume of User Datagram Protocol (UDP) packets. Unlike TCP, UDP is connectionless and stateless—the target must process each incoming UDP packet, consuming CPU and network bandwidth. The attacker sends UDP packets to random or specific ports on the victim machine, often using spoofed source IP addresses and ports. When the target responds to invalid UDP packets, additional bandwidth is consumed. In captured traffic, UDP flood attacks manifest as an exponential increase in UDP packet volume directed toward specific destination ports, coupled with high packet rates far exceeding normal baseline traffic.

d) *IGMP Spoofing (Internet Group Management Protocol Abuse)*: Internet Group Management Protocol (IGMP) spoofing exploits the multicast group membership mechanism. The attacker sends forged IGMP membership reports (e.g., IGMP v2 or v3 packets) to join multicast groups or trigger multicast traffic floods. In some configurations, this can cause the victim to receive unwanted multicast streams, consuming bandwidth and disrupting services. IGMP snooping switches may also be manipulated to forward multicast traffic to unintended ports. In PCAP analysis, this attack is identified through abnormal IGMP join/leave message patterns, rapid group membership changes, and membership reports from unauthorized source addresses.

4) *Normal Traffic Capture*: In addition to malicious traffic, benign network activity was captured to establish a statistical baseline of normal network behavior. Normal traffic included:

- HTTP/HTTPS web browsing and API requests
- DNS queries for legitimate domain resolution
- ARP requests and replies for normal host discovery
- SSH sessions for remote administration
- ICMP Echo (ping) for network diagnostics
- Multicast DNS (mDNS) for service discovery

This benign traffic was captured during regular operation of the virtual environment and labeled as "Normal" for training purposes. The presence of legitimate traffic patterns is essential for the machine learning classifier to distinguish between malicious anomalies and expected network operations, reducing false-positive detections in production deployment.

5) *Traffic Capture and Dataset Labeling*: All network traffic—encompassing benign, normal communications and all malicious attack scenarios—was captured in real-time using Wireshark in promiscuous mode on a passive network tap. The complete packet captures were stored in PCAP (Packet

Capture) binary format, preserving all layer-2 through layer-7 information for subsequent offline analysis.

Critical to the integrity of the machine learning dataset, each traffic capture was manually annotated and labeled with the corresponding attack type (or “Normal” for benign traffic). This ground-truth labeling enabled supervised learning during the model training phase. The labeled PCAP files were then exported and processed by a custom Python-based feature extraction pipeline (detailed in Section II.B) to transform raw packet data into the 35-dimensional feature vectors required for classification.

The rigorous separation between data generation (this subsection), feature engineering (Section II.B), and model development (Section III) ensures methodological clarity and reproducibility of the tIDS framework.

### B. Data Transformation & Feature Extraction

The raw PCAP and PCAPNG captures produced by the attack simulation phase cannot be directly consumed by machine learning algorithms. A dedicated data transformation pipeline was therefore engineered to convert unstructured packet streams into a structured, tabular dataset suitable for supervised classification. This pipeline is implemented in the `Convert_pcap_to_csv.py` script, which leverages the Scapy packet dissection library to iteratively parse each capture file without loading the entire trace into memory.

1) **Time-Window Aggregation:** Rather than analyzing individual packets—which would lack temporal context—the pipeline partitions each PCAP file into fixed **1-second time windows**. Every window constitutes a single data instance (row) in the output dataset. This temporal aggregation is critical because network attacks manifest as sustained behavioral patterns over time, not as isolated packets. A dictionary-based accumulator tracks per-window statistics, and upon processing all packets within a given second, the accumulated counters are finalized into a feature vector and written to the output CSV.

2) **Feature Engineering:** A total of **38 behavioral features** are extracted per time window, organized into eight semantically coherent groups:

- 1) **Traffic Volume (4 features):** *packet\_rate*, *byte\_rate*, *mean\_pkt\_size*, and *max\_pkt\_size*. These capture the overall intensity of network activity. Flooding attacks such as SYN Flood and Smurf exhibit abnormally elevated packet and byte rates, while the mean packet size tends to be small during SYN floods (due to minimal TCP headers) and large during DNS amplification.
- 2) **TCP Flag Analysis (7 features):** Raw counts of SYN, ACK, RST, and FIN flags, plus three derived ratios: *syn\_to\_ack\_ratio*, *rst\_to\_syn\_ratio*, and *tcp\_to\_total\_ratio*. The SYN-to-ACK ratio is the single most discriminative feature for SYN Flood detection; legitimate TCP traffic maintains a ratio near 1.0, whereas a SYN flood drives this ratio to extreme values (clipped at 10.0 to limit outlier influence). Pure SYN-only flags

(SYN set without ACK) are counted separately from SYN-ACK handshake responses to prevent conflation.

- 3) **UDP Metrics (2 features):** *udp\_packet\_count* and *udp\_to\_total\_ratio*. A UDP-to-total ratio approaching 1.0 is a strong indicator of UDP Flood, where the attacker overwhelms the victim with connectionless datagrams.
- 4) **ICMP Metrics (5 features):** *icmp\_packet\_count*, *icmp\_to\_total\_ratio*, *icmp\_echo\_request\_count*, *icmp\_echo\_reply\_count*, and *icmp\_reply\_to\_request\_ratio*. The Smurf attack generates massive volumes of ICMP Echo Requests directed at the broadcast address; consequently, elevated ICMP counts combined with high broadcast packet rates serve as a definitive Smurf signature.
- 5) **ARP Analysis (6 features):** *arp\_request\_count*, *arp\_reply\_count*, *arp\_reply\_ratio*, *gratuitous\_arp\_count*, *gratuitous\_to\_reply\_ratio*, and *ip\_mac\_mapping\_changes*. ARP poisoning attacks inject fraudulent ARP replies to hijack IP-to-MAC mappings. The *ip\_mac\_mapping\_changes* feature tracks instances where a previously observed IP address becomes associated with a different MAC address within the same capture session—a direct indicator of active ARP cache poisoning.
- 6) **DNS Analysis (5 features):** *dns\_query\_count*, *dns\_response\_count*, *dns\_reply\_to\_query\_ratio*, *dns\_avg\_response\_size*, and *dns\_ttl\_mean*. DNS spoofing attacks produce anomalous response-to-query ratios (clipped at 5.0) and atypical TTL values compared to legitimate resolver behavior.
- 7) **Control Protocol Features (4 features):** *stp\_packet\_count*, *stp\_topology\_change*, *igmp\_packet\_count*, and *igmp\_join\_count*. STP spoofing attacks inject forged Bridge Protocol Data Units (BPDUs) with the Topology Change flag set to destabilize the Layer-2 switching fabric. IGMP spoofing floods the network with forged Membership Report messages (types 0x12, 0x16, 0x22) targeting multicast groups. Scapy’s STP dissection is augmented with explicit LLC/Dot3 frame parsing to handle the non-IP encapsulation used by STP BPDUs.
- 8) **Network Diversity (5 features):** *unique\_src\_ip\_count*, *unique\_dst\_ip\_count*, *unique\_dst\_ports*, *broadcast\_pkt\_count*, and *multicast\_pkt\_count*. Diversity features contextualize the communication topology of each window. Distributed attacks exhibit high unique source IP counts, while Smurf attacks produce elevated broadcast counts and IGMP spoofing yields high multicast counts.

3) **Derived Ratio Features:** Several features are computed as ratios using a safe division utility that returns 0.0 when the denominator is zero, preventing division-by-zero errors. Certain ratios are additionally clipped to upper bounds (e.g., *syn\_to\_ack\_ratio*  $\leq$  10.0, *dns\_reply\_to\_query\_ratio*  $\leq$  5.0)

to mitigate the effect of extreme outlier windows on model training.

4) *Output Dataset*: The pipeline processes all eight PCAP files—one per attack class plus Normal traffic—and produces a consolidated CSV file (`tIDS_final_data.csv`) containing 9,472 labeled instances across 38 features plus a class label column. The resulting dataset exhibits reasonable class balance, with each class contributing between 1,126 and 1,226 instances, ensuring that the subsequent machine learning models are not biased toward majority classes.

### III. PROPOSED SOLUTION: ML ARCHITECTURE

The development of the tIDS centers on a behavioral machine learning pipeline designed to distinguish between legitimate network operations and malicious intrusions based on statistical traffic patterns. To achieve this, raw packet captures were processed using a 1-second time window, extracting an initial set of 38 complex mathematical and relational features (such as the `syn_to_ack_ratio` and `tcp_flags_count`). Crucially, debugging columns including the timestamp, source IP, and destination IP were deliberately excluded. This prevents the algorithms from erroneously memorizing specific IP addresses, forcing the models to learn the actual statistical behaviors of the attacks regardless of changing network topologies.

Prior to model training, a rigorous data variance audit was conducted on the extracted features. The analysis identified three features exhibiting zero variance. These static features were systematically dropped to eliminate mathematical noise and reduce computational overhead, resulting in an optimized 35-feature dataset comprising 9,472 packet records.

The dataset was subsequently divided using a 70/30 train-test split. To prevent features with inherently large magnitudes (e.g., packet rates) from disproportionately dominating features with smaller scales (e.g., specific flag counts), a *StandardScaler* was applied. This normalized all features to a mean of zero and a standard deviation of one, fitting the scaler strictly on the training data to prevent data leakage.

Three classification algorithms were selected for evaluation: Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and a Random Forest Classifier. To ensure mathematically optimal configurations for our custom traffic signatures, automated hyperparameter tuning was executed utilizing 5-fold cross-validation. A *RandomizedSearchCV* was employed for the Random Forest, locking in a maximum depth of 33 and 180 estimators. Concurrently, *GridSearchCV* identified optimal settings for KNN (`n_neighbors = 3`, `weights = distance`) and Naive Bayes (`var_smoothing = 1e-07`). The finalized models were then officially trained on the scaled training set.

#### A. Real-Time Traffic Sniffing

To move the tIDS from an offline analysis tool to something usable in practice, the PCAP-based pipeline was adapted into a real-time monitoring system. Instead of processing stored captures, the system now listens to live network traffic and produces predictions continuously. The implementation is split into three main parts: a packet capture engine, a machine

learning inference module, and a WebSocket-based server for streaming results.

1) *Capture Engine Architecture*: The real-time capture component (implemented in `sniffer.py`) is built around a *CaptureEngine* class. It runs Scapy's `sniff()` function inside a separate background thread so that packet capture does not block the rest of the application. Unlike the offline version, which reads packets from files, this setup captures packets directly from the network interface. On Windows systems, this requires installing Npcap or WinPcap.

To stay consistent with the training pipeline, the engine uses a rolling **1-second time window**. As packets arrive, their statistics are accumulated in a dictionary (packet counts, flags, byte totals, IP sets, etc.). Once one second has passed, the engine calls a `finalize_window()` method to compute the final feature values such as ratios, averages, and counts. The result is a feature vector with the same structure as the rows used during training. After that, the counters are reset and the next window starts immediately. In practice, this approach provides continuous coverage without gaps between windows.

2) *ML Inference Bridge*: Once a feature vector is generated, it is passed to the inference module (`engine.py`). This module loads the trained Random Forest model (`tIDS_RandomForest_Model.pkl`) along with its *StandardScaler* (`tIDS_Scaler.pkl`) and keeps them in memory for fast access.

One small complication is that the model was trained on 35 features, not 38, because a few features had zero variance and were removed during training. To handle this, the inference step applies a simple feature filtering step to match the expected input format. The selected features are then converted into a single-row Pandas DataFrame, scaled using the saved scaler, and passed to the model for prediction.

The whole process—from receiving the feature dictionary to producing a prediction—takes only a few milliseconds (typically under 5 ms), which is fast enough for real-time monitoring.

3) *Asynchronous Data Pipeline*: The application server (`main.py`) is built using FastAPI and runs on Uvicorn, taking advantage of Python's `asyncio` support. Since Scapy's sniffing is blocking, it runs in its own thread, while the web server remains fully asynchronous.

To safely pass data between these two parts, an `asyncio.Queue` is used. After each prediction, the capture engine pushes a JSON result into the queue using `asyncio.run_coroutine_threadsafe()`. This avoids race conditions and keeps the system stable.

On the server side, a background task (`broadcast_worker`) continuously reads from the queue and sends the results to all connected clients via WebSockets. A *ConnectionManager* handles multiple connections, allowing several users to monitor the system at the same time from different browsers.

In addition to real-time streaming, the system also provides simple REST endpoints (`/api/start` and `/api/stop`) to control the capture process without restarting the application.

## B. User Interface & Model Integration

A web-based dashboard and backend integration system were developed to provide real-time threat detection and alert visualization. The trained Random Forest model and StandardScaler were serialized as `.pkl` files and integrated into the backend server.

1) *Dashboard Interface*: The user interface provides analysts with real-time visibility of network traffic and detected anomalies. Key features include:

- **Live Traffic Monitor**: Displays captured packets with source/destination IPs, protocols, and timestamps. Detected attacks are highlighted with color-coded severity indicators.
- **Classification Results**: Shows real-time predictions (Normal or one of 7 attack types) with confidence scores.
- **Alert System**: Generates timestamped alerts for detected attacks, including attack type and source/destination information.
- **Packet Inspector**: Allows forensic analysis of individual packets and their extracted feature vectors.

2) *Model Deployment and Serialization*: The trained Random Forest classifier and fitted StandardScaler were exported using Python's `pickle` module. This preserves the model state (decision tree structures, hyperparameters, feature importance) and the scaling parameters (mean and standard deviation vectors) learned during training.

Critical to production deployment, the same scaling transformation used during training is applied to all incoming features in real-time to ensure mathematical consistency. Additionally, the feature extraction mapping (specifying which 35 features are computed and their order) is stored as a configuration file to prevent feature schema mismatches.

3) *Real-Time Detection Pipeline*: The system operates through a four-stage pipeline:

- 1) **Packet Capture**: The sniffer captures packets in promiscuous mode and queues them for processing.
- 2) **Feature Extraction**: The backend extracts the 35 behavioral features and normalizes them using StandardScaler.
- 3) **Model Inference**: The Random Forest classifier predicts the traffic class (Normal/Attack) and outputs a confidence score.
- 4) **Alert Generation**: If confidence exceeds 70% threshold, an alert is triggered and sent to the dashboard.

The architecture employs a decoupled, queue-based design: the sniffer feeds feature vectors into a message queue, the backend consumes from this queue and performs inference, and the dashboard queries results in real-time. This design enables independent scaling, fault tolerance, and easy model updates without disrupting other components.

4) *Operational Considerations*: Confidence-based thresholding (default: 70%) balances detection sensitivity and false-positive rates. This threshold was calibrated to match the 99% precision achieved on the test set, minimizing analyst alert fatigue while maintaining high detection capability. The system supports integration with external SIEM platforms

and automated response systems (e.g., firewall blocking, rate-limiting) for practical deployment in operational networks.

## IV. EVALUATION AND RESULTS

To evaluate the real-world efficacy of the trained models and test for algorithmic overfitting, the classifiers were exposed to the unseen 30% testing set (2,842 packet records). Performance was quantified using classification reports—calculating Precision, Recall, and F1-Score—and visualized via confusion matrix heatmaps.

The Gaussian Naive Bayes model established a statistical baseline, achieving an overall accuracy of 84%. While it effectively identified high-volume volumetric attacks (scoring 100% precision on Smurf and SYN floods), it struggled significantly with benign traffic, yielding only a 44% precision for the *Normal* baseline. This degradation is attributed to the algorithm's core assumption of feature independence, which mathematically falters when processing 35 highly correlated network features.

The K-Nearest Neighbors (KNN) model demonstrated a massive improvement, reaching an overall accuracy of 98%. By eliminating the zero-variance feature noise during the pre-processing phase, the geometric clustering within the feature space became highly distinct, allowing KNN to accurately categorize both brute-force and stealth attacks (such as ARP Spoofing).

Ultimately, the Random Forest Classifier emerged as the champion architecture, achieving a near-perfect 99% overall accuracy. The ensemble decision-tree structure effortlessly navigated the high dimensionality of the dataset. Most notably, the Random Forest achieved 99% Precision and 99% Recall for *Normal* network traffic. In the context of a live Security Operations Center (SOC), this near-zero false-positive rate is a critical requirement to prevent analyst alert fatigue.

While an accuracy score of 99% frequently suggests algorithmic overfitting, the model maintained this exceptional performance on the isolated holdout test set and through the 5-fold cross-validation phase. This confirms that the algorithm successfully learned the generalized attack signatures rather than merely memorizing the training data.

However, we acknowledge an element of "environmental overfitting" inherent to the dataset. Because the custom VirtualBox lab was air-gapped, it lacked the chaotic background noise typical of a live enterprise environment. Consequently, the algorithms were trained on mathematically pure examples of cyberattacks. To deploy the tIDS into a production environment, the exported machine learning pipeline (`.pkl` files) would require retraining on a noisier, enterprise-level dataset, though the foundational architecture and relational feature engineering remain highly robust.

## V. CONCLUSION

This project successfully demonstrates the critical viability of transitioning from traditional, signature-based network defenses to dynamic, behavioral anomaly detection. The development of the tiny Intrusion Detection System (tIDS) provided

an end-to-end framework for identifying sophisticated link-layer, network-layer, and transport-layer threats. By engineering a 35-feature dataset derived from simulated, live-fire attacks—specifically targeting STP, ARP, and DNS spoofing, alongside TCP SYN, Smurf, IGMP, and UDP floods—the system established a highly robust statistical baseline for anomalous network traffic. The seamless integration of a custom real-time packet sniffer with an interactive front-end dashboard, powered by an optimized Random Forest classifier, culminated in an exceptional 99% detection accuracy with near-zero false positives.

Despite these highly promising results, several critical avenues remain for future improvement. Because the tIDS was developed, trained, and evaluated within a sterile, air-gapped laboratory environment, the current machine learning pipeline is inherently optimized for mathematically pure attack signatures. To achieve production readiness, future iterations must prioritize exposing the real-time sniffer to noisy, enterprise-level network traffic to evaluate the model’s resilience against benign background interference. Retraining the exported pipeline on diverse, non-synthetic datasets will further harden the architecture against false positives. Additionally, deploying the tIDS as a localized monitoring tool within a live campus network would provide invaluable insights into the operational latency between the backend sniffer, the algorithmic processing engine, and the front-end alert system under real-world stress.

#### ACKNOWLEDGMENT

The authors would like to thank Dr. Karim Lounis for his guidance throughout the Computer and Network Security course.